

Chapter 6: Planner–Executor Pattern and Hierarchical Task Decomposition

Part II: Agentic Architecture Patterns

Book: Agentic AI & RAG Systems Handbook

Version: 1.0.0

Chapter ID: 06

File Name: chapter-06-planner-executor-pattern.md

Chapter Overview

The ReAct pattern works extremely well for short and moderately complex tasks.

However, as objectives become larger, ReAct begins to struggle.

Example:

1 Create a due diligence report for a company acquisition.

Such a task may require:

- Financial analysis
- Competitive analysis
- Legal review
- Risk assessment
- Executive summary creation

A single ReAct loop quickly becomes inefficient.

The Planner–Executor Pattern solves this problem by separating:

1 Thinking About Work

from

1 Doing Work

This separation is one of the most important principles in enterprise agent architecture and appears repeatedly in:

- LangGraph
- CrewAI
- AutoGen
- Semantic Kernel planners

- Multi-agent systems
 - Agentic operations platforms
-

Learning Objectives

After completing this chapter you should be able to:

- Understand the Planner–Executor architectural model
 - Design task decomposition strategies
 - Implement hierarchical planning systems
 - Build planner-executor workflows in LangGraph
 - Scale planners across multi-agent systems
 - Handle execution failures properly
 - Govern planning systems
 - Measure planning effectiveness
 - Apply Planner–Executor in enterprise architectures
-

Prerequisites

Required:

- Chapter 1: Generative AI Foundations
 - Chapter 2: Agentic AI Fundamentals
 - Chapter 3: Agent Design Principles
 - Chapter 4: ReAct Pattern
 - Chapter 5: Router Pattern
-

1. Introduction

Consider the request:

1 Prepare a cloud modernization strategy.

A ReAct agent may repeatedly reason and execute:

1 Reason
2 ↓
3 Search

```
4   ↓
5   Reason
6   ↓
7   Search
8   ↓
9   Reason
```

This creates:

- Long execution times
- Excessive tokens
- Little structure
- Difficult governance

The Planner–Executor pattern first creates a plan:

```
1   Goal
2   ↓
3   Plan
4   ↓
5   Tasks
6   ↓
7   Execution
8   ↓
9   Aggregation
10  ↓
11  Result
```

The planner determines *what should be done*.

The executor determines *how tasks are completed*.

2. Core Concepts

What Is The Planner–Executor Pattern?

Definition

The Planner–Executor pattern separates:

Role	Responsibility
Planner	Determines tasks
Executor	Performs tasks
Aggregator	Combines outputs

Core Principle

- | |
|----------------|
| 1 Plan Once |
| 2 Execute Many |

Instead of repeatedly deciding:

- | |
|--------------------------|
| 1 What should I do next? |
|--------------------------|

the system first builds a roadmap.

3. Why This Pattern Exists

Without planning:

- | |
|-----------|
| 1 Goal |
| 2 ↓ |
| 3 Actions |
| 4 ↓ |
| 5 Actions |
| 6 ↓ |
| 7 Actions |

Actions may become:

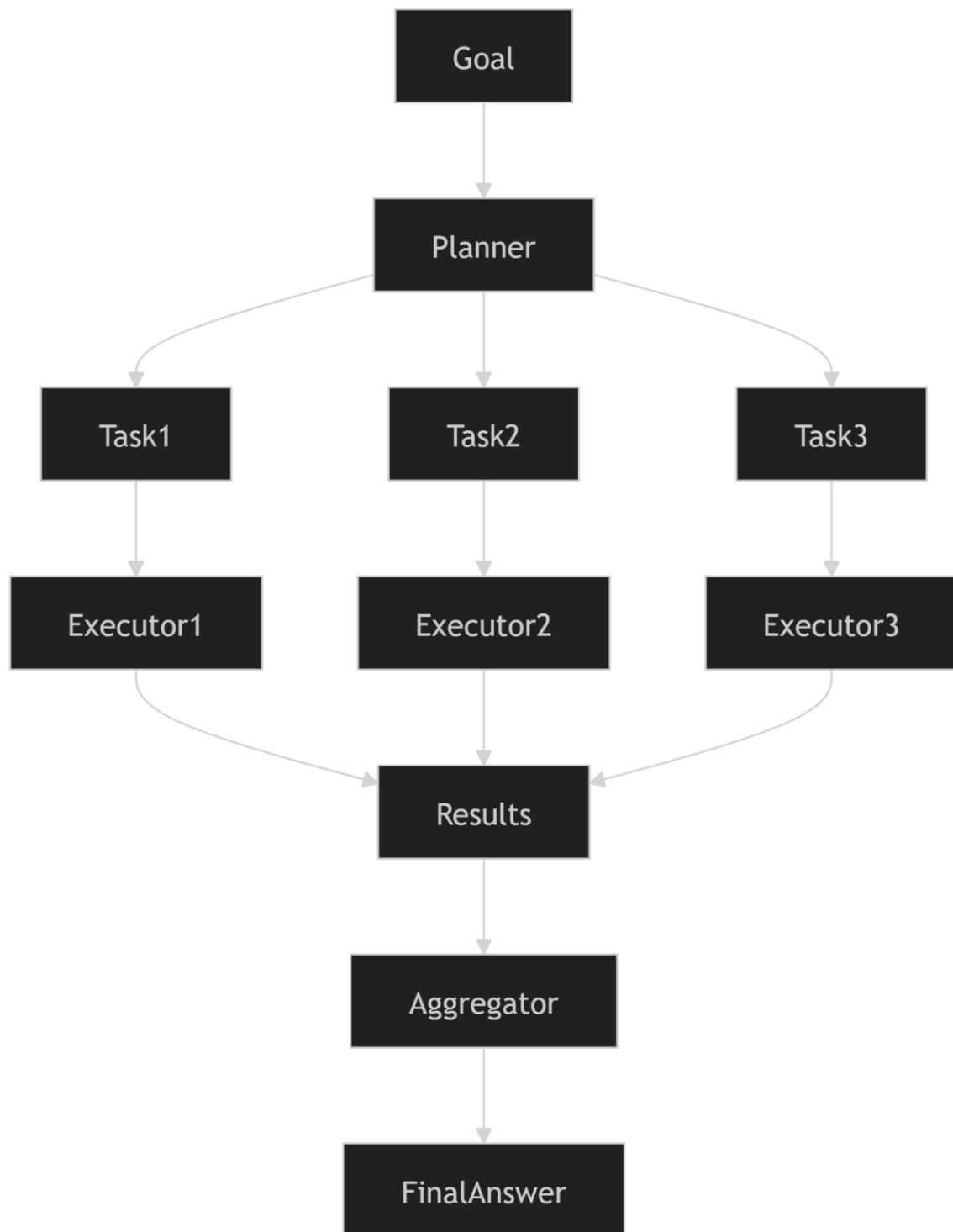
- Redundant
- Inefficient
- Expensive

With planning:

- | |
|-----------|
| 1 Goal |
| 2 ↓ |
| 3 Plan |
| 4 ↓ |
| 5 Execute |
| 6 ↓ |
| 7 Verify |

The process becomes structured.

4. High-Level Architecture



5. Task Decomposition

What Is Task Decomposition?

Large goals are divided into smaller manageable tasks.

Example

Goal:

1 Perform customer churn analysis.

Planner output:

1 Task 1: Gather customer metrics
2
3 Task 2: Analyze churn drivers
4
5 Task 3: Create executive summary

Another Example

Goal:

1 Assess cloud migration readiness.

Planner output:

1 Infrastructure Assessment
2
3 Application Assessment
4
5 Security Assessment
6
7 Dependency Analysis

6. Planner Responsibilities

Planners perform:

Goal Analysis

1 What is being requested?

Scope Definition

1 How much work is required?

Task Generation

1 Break into tasks.

Dependency Analysis

1 Which tasks depend on others?

Prioritization

1 What should happen first?

7. Executor Responsibilities

Executors should not plan.

Executors should execute.

Typical Executor Functions

Retrieval

1 Search documents

Analysis

1 Analyze data

Tool Usage

1 Call APIs

Transformations

1 Generate reports

Validation

1 Verify outputs

8. Planning Strategies

Strategy 1: Linear Planning

1	A
2	↓
3	B
4	↓
5	C

Simple but slower.

Strategy 2: Parallel Planning

1	Task A
2	Task B
3	Task C

Execute simultaneously.

Strategy 3: Hierarchical Planning

1	Goal
2	↓
3	Subgoal
4	↓
5	Tasks

Ideal for complex enterprise work.

Strategy 4: Adaptive Planning

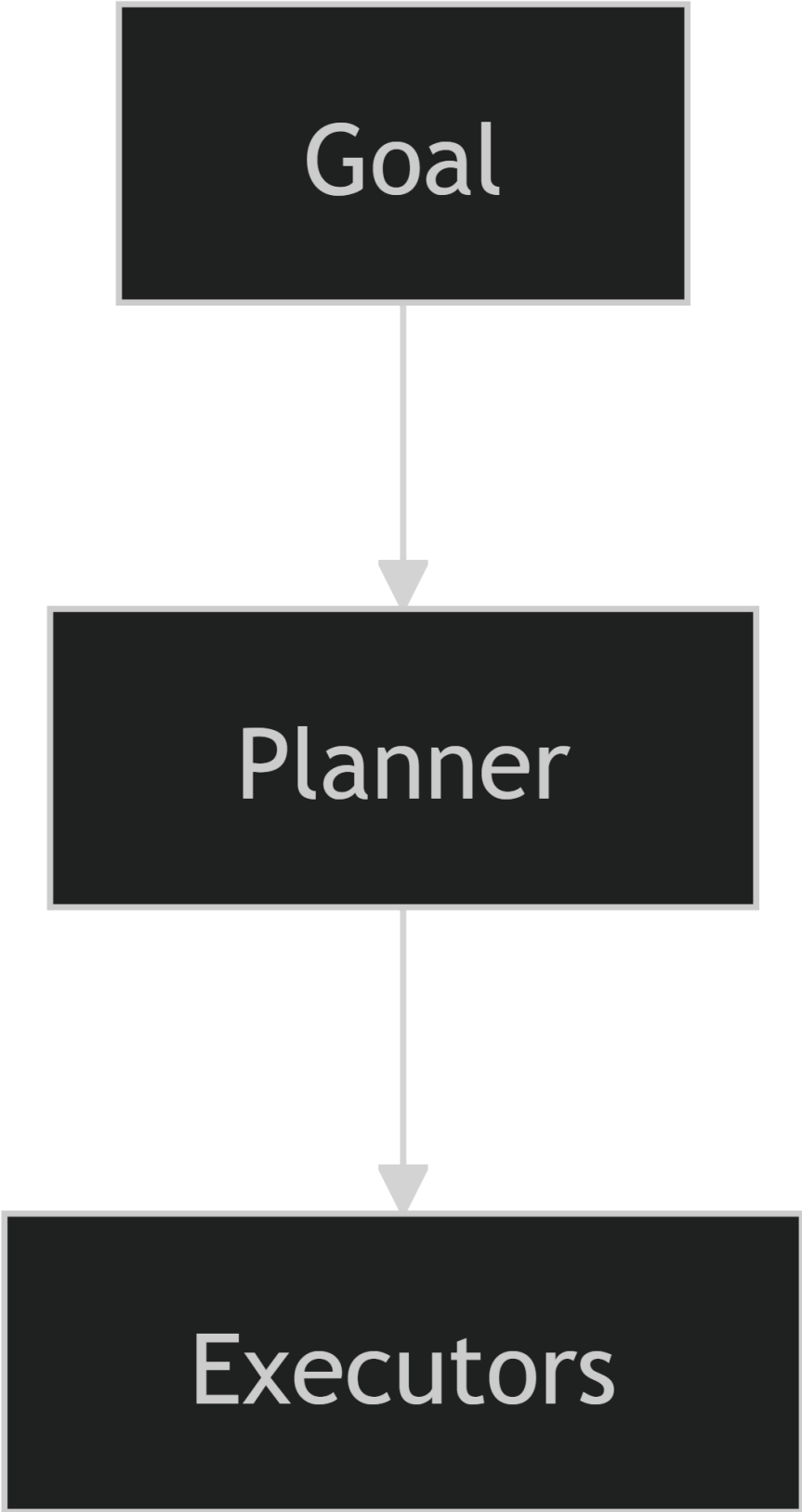
Plan may be updated as execution progresses.

1	Plan
2	↓
3	Observe
4	↓
5	Replan

9. Planner Types

Single Planner

Goal



```
graph TD; Goal[Goal] --> Planner[Planner]; Planner --> Executors[Executors];
```

Planner

Executors

Best for moderate complexity.

Multi-Level Planner

Goal

```
graph TD; Goal[Goal] --> StrategicPlanner[StrategicPlanner]; StrategicPlanner --> TacticalPlanner[TacticalPlanner]; TacticalPlanner --> OperationalTasks[OperationalTasks];
```

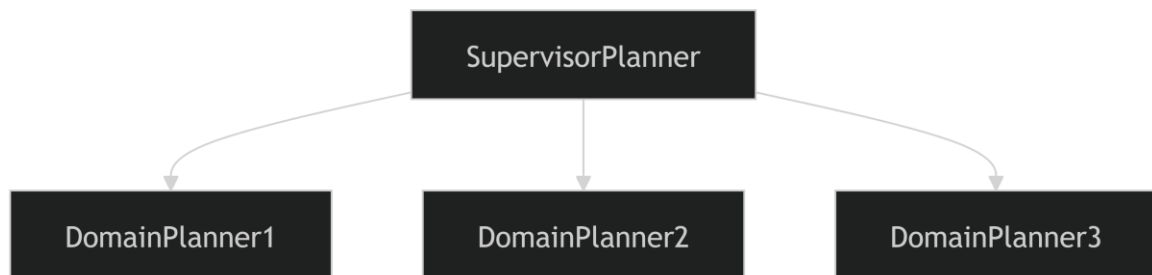
StrategicPlanner

TacticalPlanner

OperationalTasks

Common in digital workforce architectures.

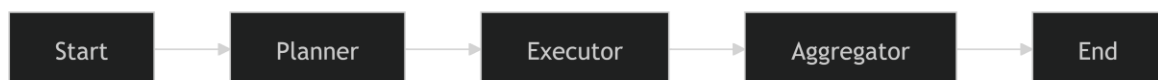
Planner Supervisor



Used in enterprise-scale systems.

10. LangGraph Implementation

Graph Structure



State Model

```
1 from typing import TypedDict
2
3 class PlanState(TypedDict):
4     goal: str
5     tasks: list
6     results: list
7     final_answer: str
```

Planner Node

```
1 def planner(state):
2     return {
3         "tasks": [
4             "Retrieve financial reports",
```

```
5         "Analyze risks",
6         "Generate summary"
7     ]
8 }
```

Executor Node

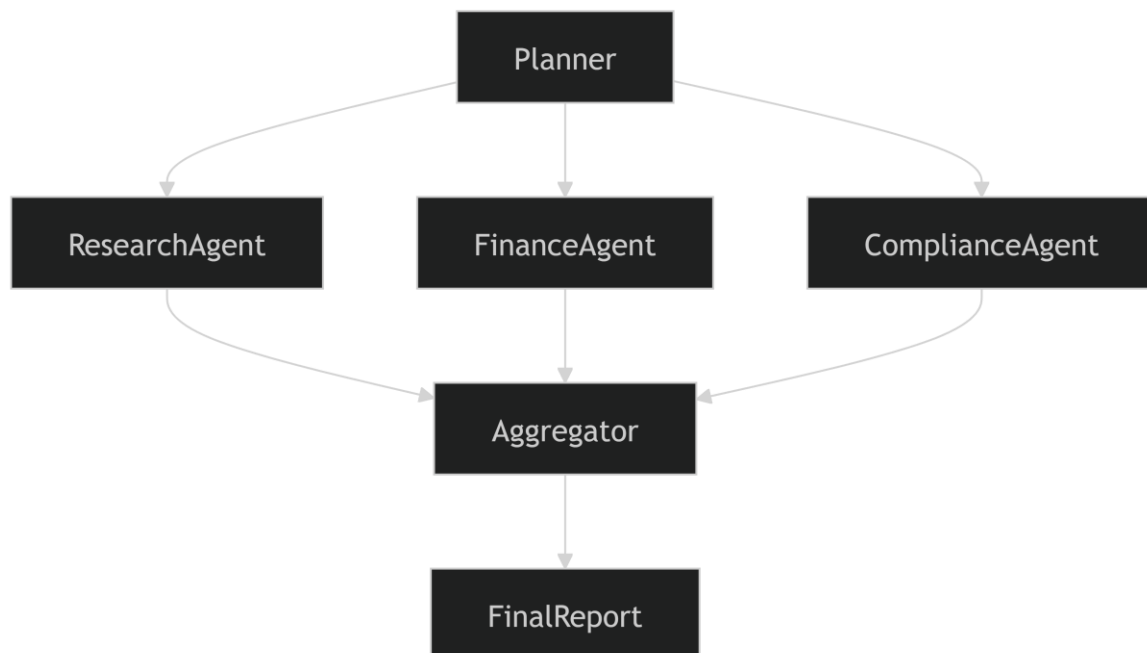
```
1 def executor(task):
2     return execute(task)
```

Aggregator Node

```
1 def aggregate(results):
2     return combine(results)
```

11. Multi-Agent Planner–Executor

Multi-agent systems frequently use this architecture.



12. Planner–Executor + Router

A common enterprise architecture.

1	Goal
2	↓
3	Planner
4	↓
5	Router
6	↓
7	Best Executor

The planner creates the task.

The router selects the resource.

13. Planner–Executor + ReAct

The most common production design.

1	Planner
2	↓
3	Task
4	
5	Executor
6	↓
7	ReAct Loop

Planning occurs once.

Reasoning occurs inside execution.

14. CrewAI Architecture

CrewAI frequently implements:

1	Manager Agent
2	↓
3	Worker Agents

Equivalent:

1	Planner
---	---------

- | | |
|---|-----------|
| 2 | ↓ |
| 3 | Executors |

15. AutoGen Architecture

AutoGen commonly supports:

- | | |
|---|-------------|
| 1 | Coordinator |
| 2 | ↓ |
| 3 | Specialists |

Planner–Executor is a frequent AutoGen pattern.

16. Semantic Kernel Planning

Semantic Kernel's planner architecture is explicitly aligned with:

- | | |
|---|-----------------|
| 1 | Goal |
| 2 | ↓ |
| 3 | Plan |
| 4 | ↓ |
| 5 | Skill Selection |
| 6 | ↓ |
| 7 | Execution |

17. Enterprise Due Diligence Example

Goal:

- | | |
|---|------------------------------|
| 1 | Evaluate acquisition target. |
|---|------------------------------|

Planner:

- | | |
|---|------------------|
| 1 | Financial Review |
| 2 | |
| 3 | Legal Review |
| 4 | |
| 5 | Security Review |
| 6 | |
| 7 | Market Review |

Specialists execute independently.

Aggregator produces final recommendation.

18. RAG Planning Example

Goal:

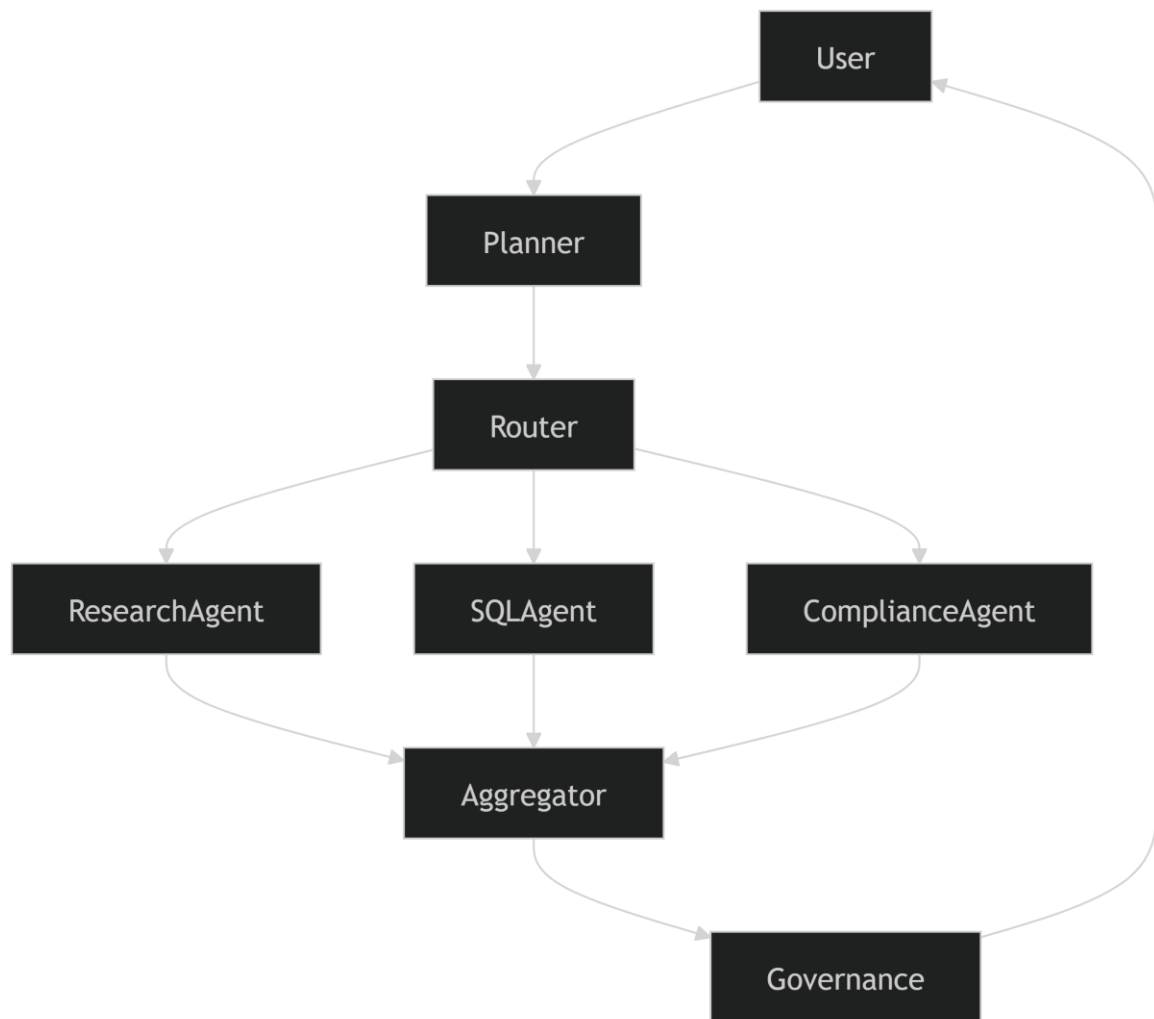
1 Prepare policy impact assessment.

Plan:

1 Retrieve policies
2
3 Retrieve regulations
4
5 Map dependencies
6
7 Generate report

Each task becomes independently executable.

19. Production Architecture



20. Scalability

Planner systems scale through:

Task Parallelism

1	100 Tasks
---	-----------

running simultaneously.

Agent Parallelism

1	10 Agents
---	-----------

executing at once.

Workflow Distribution

1 Task Queues

supporting large workloads.

21. Reliability

Common controls:

- Retry policies
 - Failure recovery
 - Task replay
 - Approval checkpoints
 - Validation gates
-

Example

1 <code>max_retries: 3</code>
2
3 <code>timeout: 120s</code>

22. Security

Planning increases risk because planning can accidentally generate unsafe tasks.

Example:

1 Delete records

Planners require:

- Policy validation
- Risk assessment
- Permission checks

Architectural Principle:

1 Planning
2 ≠
3 Authorization

23. Observability

Track:

Metric	Purpose
Plan Quality	Planning effectiveness
Tasks Per Goal	Complexity
Task Success Rate	Execution quality
Execution Latency	Performance
Replans	Stability
Cost Per Plan	FinOps

24. Failure Modes

Failure 1

Bad Plan

1 Wrong decomposition

Failure 2

Too Many Tasks

1 Complexity explosion

Failure 3

Missing Dependencies

Execution order becomes invalid.

Failure 4

Executor Drift

Tasks interpreted incorrectly.

Failure 5

Aggregation Failure

Final synthesis becomes inaccurate.

25. Common Mistakes

Mistake 1

Using a planner for simple work.

Mistake 2

Excessive task decomposition.

Mistake 3

Allowing executors to replan everything.

Mistake 4

No validation layer.

Mistake 5

No aggregation strategy.

26. Anti-Patterns

Infinite Planning

1	Planning
2	↓
3	Planning
4	↓
5	Planning

Never executes.

Micro-Task Explosion

- | | |
|---|------------|
| 1 | 1 Goal |
| 2 | ↓ |
| 3 | 1000 Tasks |

Expensive.

Replanning Everything

Wastes resources.

Hidden Dependencies

Tasks fail unpredictably.

27. Trade-Offs

Advantage	Cost
Better organization	More complexity
Better scalability	More orchestration
Better specialization	More coordination
Better governance	More architecture

28. Architecture Decision Matrix

Scenario	Planner-Executor Fit
Research Platform	Excellent
Due Diligence	Excellent
Compliance Reviews	Excellent
Multi-Agent Systems	Excellent
IT Operations	Excellent
FAQ Bot	Poor
Simple Search	Poor

29. Real-World Use Cases

Research Intelligence Platform

Parallel investigative tasks.

Compliance Programs

Control assessment workflows.

Enterprise Copilot

Complex request decomposition.

Digital Workforce

Supervisor planning and specialist execution.

Agentic Operations Platform

Task orchestration and execution.

30. Interview Questions

Q1

What is the Planner–Executor Pattern?

Q2

Why separate planning from execution?

Q3

When should this pattern be used?

Q4

How does it improve scalability?

Q5

How does it differ from ReAct?

Q6

What are the primary risks?

Q7

How does LangGraph support Planner–Executor?

Q8

What is the strongest enterprise architecture?

31. Interview Answers

Q1

A pattern that separates task decomposition from task execution.

Q2

It improves scalability, governance, specialization, and observability.

Q3

For multi-step, complex, uncertain tasks.

Q4

Tasks can execute independently and in parallel.

Q5

ReAct decides one step at a time; Planner–Executor builds an upfront task structure.

Q6

Poor planning quality, excessive decomposition, and orchestration complexity.

Q7

Through planner nodes, executor nodes, conditional routing, and state management.

Q8

Planner + Router + Specialist Executors + Aggregation + Governance.

32. Architecture Review Checklist

Planner–Executor Readiness

- Planning strategy defined
- Dependency model documented
- Execution strategy defined
- Routing strategy implemented

- Aggregation process designed
 - Governance controls applied
 - Observability metrics captured
 - Cost controls implemented
 - Failure handling documented
-

33. Practical Exercises

1. Design a due diligence Planner–Executor system.
 2. Implement a LangGraph planner workflow.
 3. Compare ReAct and Planner–Executor architectures.
 4. Design aggregation logic for a multi-agent platform.
 5. Build governance controls for planner-generated tasks.
-

34. Key Takeaways

- Planner–Executor separates thinking from doing.
 - Complex goals should be decomposed into manageable tasks.
 - Parallel execution improves scalability dramatically.
 - Planning, routing, execution, and aggregation are distinct concerns.
 - Multi-agent systems frequently rely on Planner–Executor architectures.
 - LangGraph naturally supports planner-based orchestration.
 - Planning does not imply authorization.
 - Governance remains mandatory.
 - Poor task decomposition is the most common failure mode.
 - Planner–Executor is one of the most important production patterns in enterprise Agentic AI.
-

35. References

- Plan-and-Solve Prompting Research
- ReAct Paper
- LangGraph Documentation
- CrewAI Documentation
- AutoGen Documentation

- Semantic Kernel Documentation
 - Enterprise Multi-Agent Architecture Patterns
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 7 – Reflection and Self-Critique Pattern for Adaptive Agent Improvement